

STRUKDAT BAB 6

Queue

1. Pengertian

Queue adalah struktur data linear yang menggunakan prinsip:

FIFO

(First In First Out)

Data yang pertama masuk akan menjadi data pertama yang keluar.

Contoh:

Enqueue 10

Enqueue 20

Enqueue 30

Hasil:

FRONT

REAR

|

|

v

v

10 -> 20 -> 30

Jika dilakukan:

Dequeue

Maka:

10

akan keluar terlebih dahulu.

2. Operasi Dasar Queue

Operasi utama:

Enqueue
Dequeue
Front
Rear
IsEmpty
IsFull

3. Ilustrasi Enqueue

Awal:
Kosong

Enqueue:

10
Hasil:
FRONT
|
v

10

^
|
REAR

Enqueue:

20
Hasil:

FRONT	REAR
v	v

10 -> 20

4. Ilustrasi Dequeue

Awal:

FRONT	REAR
-------	------

10 -> 20 -> 30

Dequeue:

10

Hasil:

FRONT	REAR
-------	------

20 -> 30

5. Implementasi Queue Menggunakan Array

Deklarasi:

```
#define MAX 1000
```

```
int queue[MAX];
```

```
int front = -1;
```

```
int rear = -1;
```

Kondisi kosong:

```
front == -1
```

6. Enqueue Array

```
void enqueue(int x){
```

```
    if(rear == MAX-1){
```

```
        return;
```

```
    }
```

```
    if(front == -1){
```

```
        front = 0;
```

```
    }
```

```
    queue[++rear] = x;
```

```
}
```

7. Dequeue Array

```
int dequeue(){  
  
    if(front == -1){  
  
        return -1;  
  
    }  
  
    int x = queue[front];  
  
    if(front == rear){  
  
        front = rear = -1;  
  
    }  
  
    else{  
  
        front++;  
  
    }  
  
    return x;  
  
}
```

8. Front

```
int getFront(){  
  
    if(front == -1){  
  
        return -1;
```

```
    }  
  
    return queue[front];  
  
}
```

9. Rear

```
int getRear(){  
  
    if(rear == -1){  
  
        return -1;  
  
    }  
  
    return queue[rear];  
  
}
```

10. Traversal Queue

```
for(int i=front;i<=rear;i++){  
  
    printf("%d ",queue[i]);  
  
}
```

11. Circular Queue

Masalah Queue Array Biasa:

Dequeue terus menerus

menyebabkan ruang kosong di depan tidak bisa digunakan kembali.

Solusi:

Circular Queue

Ilustrasi:

0 1 2 3 4

Ketika mencapai akhir

kembali ke awal

Rumus:

$(\text{index} + 1) \% \text{MAX}$

12. Struktur Circular Queue

$\text{front} = (\text{front} + 1) \% \text{MAX};$

```
rear = (rear + 1) % MAX;
```

13. Implementasi Queue Menggunakan Linked List

Node:

```
struct Node{  
  
    int data;  
  
    struct Node* next;  
  
};
```

Pointer:

```
struct Node* front = NULL;  
struct Node* rear = NULL;
```

14. Enqueue Linked List

```
void enqueue(int x){  
  
    struct Node* newNode =  
        (struct Node*)malloc(sizeof(struct Node));  
  
    newNode->data = x;  
  
    newNode->next = NULL;
```



```
if(rear == NULL){  
    front = rear = newNode;  
    return;  
}  
  
rear->next = newNode;  
  
rear = newNode;  
  
}
```

15. Dequeue Linked List

```
int dequeue(){  
  
    if(front == NULL){  
        return -1;  
    }  
  
    struct Node* temp = front;  
  
    int x = temp->data;  
  
    front = front->next;  
  
    if(front == NULL){  
        rear = NULL;  
    }  
}
```

```
    free(temp);  
  
    return x;  
  
}
```

16. Aplikasi Queue

Antrian Bank

Nasabah pertama
dilayani pertama

Sistem Tiket

Pembeli pertama
dilayani pertama

Printer

Dokumen pertama
dicetak pertama

BFS

Pada tree dan graph.

17. Kompleksitas

Operasi	Array	Linked List
Enqueue	$O(1)$	$O(1)$
Dequeue	$O(1)$	$O(1)$
Front	$O(1)$	$O(1)$
Rear	$O(1)$	$O(1)$
Search	$O(n)$	$O(n)$

18. Kesalahan Umum

Queue Overflow

Enqueue ketika penuh.

Queue Underflow

Dequeue ketika kosong.

Front Tidak Diperbarui

Setelah dequeue.

Rear Tidak Diperbarui

Saat enqueue.

Memory Leak

Linked list queue:

`dequeue()`

tetapi node tidak:

`free()`

19. Implementasi Lengkap

```
#include <stdio.h>
```

```
#define MAX 100
```

```
int queue[MAX];
```

```
int front = -1;
```

```
int rear = -1;
```

```
void enqueue(int x){
```

```
    if(rear == MAX-1)
        return;
```

```
    if(front == -1)
        front = 0;
```

```
    queue[++rear] = x;
```

```
}
```

```
int dequeue(){

    if(front == -1)
        return -1;

    int x = queue[front];

    if(front == rear)
        front = rear = -1;

    else
        front++;

    return x;

}

int main(){

    enqueue(10);
    enqueue(20);
    enqueue(30);

    printf("%d\n",dequeue());

    return 0;

}
```
